بسمه تعالی



بهینه سازی ارسال داده در ارتباطات بین میکروسرویس ها در معماری رویداد محور با استفاده از مدل های ترکیبی یادگیری ماشین

دانشجو: محمد امین محمدی تبار

پروژه دوره یادگیری ماشین با پایتون

مدرس: جناب آقای مهندس مصباح

مهرماه ۱۴۰۵

فهرست مطالب

۱ –مقدمه و بیان مسئله ۴

۱–۱ –مقدمه ۴

۱–۲– بیان مختصری از پیشینه ی پژوهش ۴

۲– جمع آوری داده و روش پیشنهادی ۶

۳ –پیش‌پردازش داده ۷

۳.۱ –تعریف ویژگی‌ها و متغیر هدف ۷

۳.۲ –میانگین‌گیری از تکرار آزمایش‌ها ۷

۳.۳ –تقسیم داده به مجموعه آموزشی و آزمایشی ۸

۳.۴ –کدگذاری ویژگی دسته‌ای ۸

۳.۵ –نرمال‌سازی ویژگی‌های عددی ۸

۳.۶ –جلوگیری از نشت داده ۸

۳.۷– نرمال‌سازی متغیر هدف برایGPR ۹

۴–جنگل تصادفی ۹

۴–۱– مبانی نظری ۹

۴–۲ –پیاده‌سازی و تنظیم پارامترها ۱۰

۴–۳ –ارزیابی مدل ۱۱

۵ –XGBoost ۱۲

۵–۱– مبانی نظری ۱۲

۵.۲ پیاده‌سازی و تنظیم پارامترها ۱۳

۵–۳ –ارزیابی مدل ۱۴

۷– LightGBM ۱۷

۷–۱ –مبانی نظری ۱۷

۲-۷- پیاده‌سازی و تنظیم پارامترها ۱۸
۳-۷- ارزیابی مدل ۱۹
۸-Stacking ۱۹
۱-۸ -مبانی نظری ۱۹
۲-۸- انتخاب مدل‌های پایه ۲۰
۳-۸ -پیاده‌سازی و تنظیم پارامترها ۲۱
۴-۸- ارزیابی مدل ۲۲
۹- نتایج ۲۲
۱-۹ -مقایسه عملکرد مدل‌ها ۲۲
۲-۹ -تحلیل نتایج ۲۳
۱۰- نتیجه‌گیری ۲۴
منابع و مآخذ ۲۵

۱ -مقدمه و بیان مسئله

۱-۱ -مقدمه

امروزه با توسعه ی سیستم های کامپیوتری، حجم بسیار بالایی از داده ها تولید می شوند که این داده ها نیازمند پردازش هستند. یکی از عوامل مهم در افزایش کیفیت و عملکرد این سیستم ها، سرعت بالای سیستم ها است که یکی از فاکتور های سنجش این مهم میزان عبوردهی داده ها است. عبوردهی را در واقع می توان تعداد رکورد های جابجا شده در واحد زمان تعریف کرد. با افزایش عبوردهی سرعت کلی سیستم افزایش یافته و داده های جابجا شده سریع تر به سیستم هدف انتقال پیدا کرده و مورد پردازش قار می گیرند و در نتیجه راندمان کلی سیستم بالا می رود.

با گسترش روزافزون سامانه‌های توزیع‌شده، اینترنت اشیا، کلان‌داده و کاربردهای نیازمند پردازش بلادرنگ، معماری‌های سنتی مبتنی بر ارتباطات همگام (Synchronous) با محدودیت‌هایی نظیر مقیاس‌پذیری پایین، وابستگی شدید اجزای سیستم و افزایش تأخیر مواجه شده‌اند. در این راستا، معماری‌های رویدادمحور (Event-Driven Architecture) و سامانه‌های پیام‌رسان توزیع‌شده به عنوان راهکاری مؤثر برای توسعه سامانه‌های مقیاس‌پذیر و با قابلیت پردازش بلادرنگ مطرح شده‌اند.

یکی از مهم‌ترین اجزای این معماری‌ها، سامانه‌های صف پیام (Message Queue Systems) هستند که وظیفه انتقال، ذخیره‌سازی و توزیع رویدادها را میان سرویس‌های مختلف بر عهده دارند. در سال‌های اخیر، ابزارهایی نظیر Apache Kafka، Apache Pulsar، ActiveMQ Artemis و RocketMQ به طور گسترده در معماری‌های میکروسرویسی مورد استفاده قرار گرفته‌اند.

۱-۲- بیان مختصری از پیشینه ی پژوهش

مطالعه‌ای که توسط Chy و همکاران[1] در سال ۲۰۲۳ انجام شد، به ارزیابی مقایسه‌ای چهار سامانه پیام‌رسان مبتنی بر ماشین مجازی جاوا شامل Kafka ، Pulsar، Artemisو RocketMQ پرداخت. هدف اصلی این پژوهش بررسی عملکرد این سامانه‌ها از دیدگاه معیارهای کلیدی شامل تأخیر(Latency) ، توان عملیاتی(Throughput) ، مصرف پردازنده، مصرف حافظه و عملکرد Garbage Collection بود. نتایج نشان داد که Artemis کمترین میزان تأخیر را ارائه می‌دهد، در حالی که Kafka و Pulsar توان عملیاتی بسیار بالاتری در پردازش حجم زیاد پیام‌ها دارند. همچنین Kafkaبا وجود ارائه عملکرد پایدار، مصرف منابع سخت‌افزاری بیشتری نسبت به سایر گزینه‌ها دارد. نویسندگان نتیجه گرفتند که انتخاب سامانه پیام‌رسان باید متناسب با نیازهای کاربردی، محدودیت‌های منابع و الزامات عملکردی هر سامانه صورت گیرد. علاوه بر این، آن‌ها نیاز به تحقیقات بیشتر در زمینه مقیاس‌پذیری، امنیت و بهینه‌سازی عملکرد این سامانه‌ها را مطرح کردند .

در پژوهشی دیگر، Khriji و همکاران[2] در سال ۲۰۲۲ معماری ابری رویدادمحوری با نام REDA را برای پردازش بلادرنگ داده‌های حاصل از شبکه‌های حسگر بی‌سیم ارائه کردند. در این معماری، پروتکل MQTT برای جمع‌آوری داده‌ها از گره‌های حسگر، Apache Kafkaبه عنوان هسته پردازش جریان داده و الگوی میکروسرویس برای پیاده‌سازی منطق کسب‌وکار مورد استفاده قرار گرفت. پژوهشگران با استقرار خوشه Kafka در چند ناحیه دسترس‌پذیری و بهره‌گیری از Kafka Streamsتوانستند بستری با قابلیت تحمل خطا، مقیاس‌پذیری بالا و تأخیر کم ایجاد کنند. نتایج آزمایش‌ها نشان داد که معماری پیشنهادی قادر است بیش از ۸۰۰۰ پیام در ثانیه را با تأخیر پایین پردازش کند و در عین حال هزینه استقرار نسبتاً کمی داشته باشد .

در معماری REDA[3] ، نویسندگان بر اهمیت استفاده از الگوی Publish/Subscribe تأکید کرده‌اند. در این معماری، MQTT نقش تولیدکننده رویداد، Kafka نقش کانال رویداد و میکروسرویس‌ها نقش مصرف‌کننده رویداد را بر عهده دارند. این ساختار باعث کاهش وابستگی میان سرویس‌ها، افزایش قابلیت توسعه‌پذیری و بهبود تحمل خطا در سامانه می‌شود. همچنین برای بهینه‌سازی عملکرد Kafka ، راهکارهایی نظیر افزایش تعداد پارتیشن‌ها، استفاده از تکنیک Batching و توسعه موازی مصرف‌کننده‌ها ارائه شده است که موجب افزایش چشمگیر توان عملیاتی سیستم می‌شود .

مطالعه‌ای که توسط محمدی تبار و مقسمی[4] در سال ۲۰۲۴ انجام شد، نشان داد که استفاده از Apache Kafka برای انتقال جریان داده ها در سیستم های کلان داده در معماری میکروسرویس نسبت به ابزارهای مشابه مثل RabbitMQ و Redis مناسب تر بوده و با افزایش میزان تحمل خطا، مقیاس پذیری بیشتر برای چنین سامانه هایی کارایی بالاتری دارد.

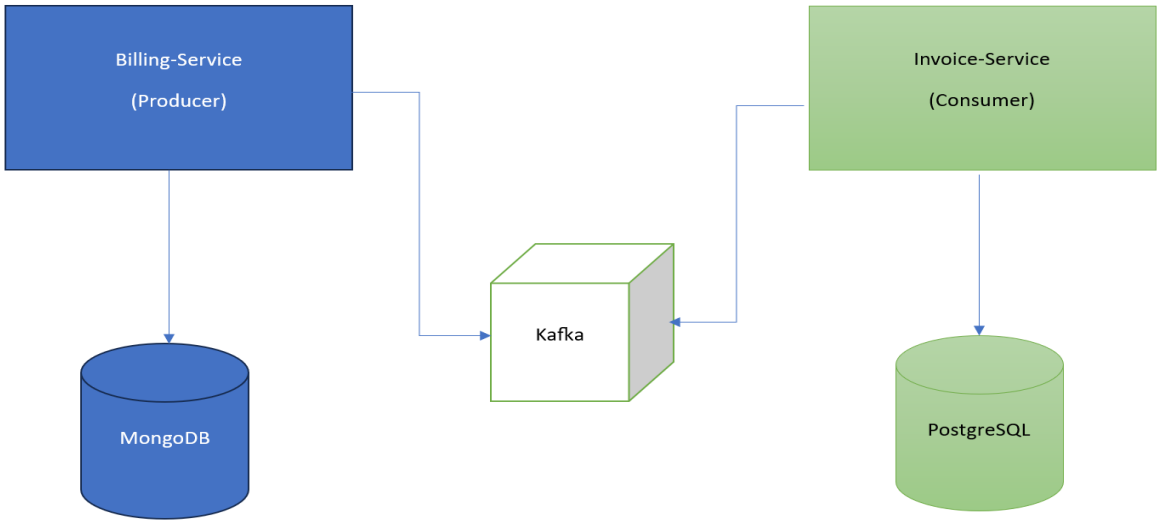
بررسی این پژوهش ها نشان می‌دهد که Kafka به عنوان یکی از مهم‌ترین فناوری‌های پردازش جریان داده، هم از منظر عملکردی و هم از منظر معماری قابلیت‌های قابل توجهی برای توسعه سامانه‌های بلادرنگ دارد. با این حال، بیشتر مطالعات موجود یا بر مقایسه عملکرد پیام‌رسان‌ها تمرکز داشته‌اند و یا صرفاً به ارائه یک معماری کاربردی در حوزه اینترنت اشیا پرداخته‌اند. در بسیاری از این پژوهش‌ها، موضوع طراحی و پیاده‌سازی یک معماری میکروسرویسی کامل مبتنی بر Kafka همراه با ارزیابی عملی شاخص‌هایی نظیر تأخیر، توان عملیاتی، مقیاس‌پذیری و تحمل خطا کمتر مورد توجه قرار گرفته است.

لذا در این پژوهش بر این شدیم تا با ارائه ی راهکاری با بهینه سازی پارامتر های پیکره بندی کارگزار پیام، روشی را برای بهینه سازی عملکرد سیستم ها ارائه نماییم.

۲- جمع آوری داده و روش پیشنهادی

در این پژوهش با طراحی و پیاده سازی یک محیط شبیه سازی تولید جریان داده ها با معماری میکروسرویس و معماری رویداد محور، داده های تولید شده در برنامه ی تولید کننده ی داده به کارگزار پیام ارسال می شود و پس از دریافت داده ها توسط کارگزار پیام، گروه های دریافت کننده که می توانند از این داده ها استفاده کنند. در این پژوهش ما قصد داریم با بهینه سازی پارامتر های کارگزار پیام به هنگام ارسال پیام توسط برنامه ای که داده ها را به کارگزار پیام ارسال می کند، عملکرد سیستم را از طریق ارتقا تعداد پیام های ارسال شده رد واحد زمان که با نام throughput می شناسیم، ارتقا دهیم.

در روش پیشنهادی، برنامه ی تولید کننده ی داده که با نام Producer شناخته می شود، داده هایی را که می خواهد به برنامه ی مصرف کننده ی داده که با عنوان Consumer شناخته می شود، ابتدا به کارگزار پیام ارسال می کند تا برنامه ی مصرف کننده ی داده به صورت غیرهمزمان یا Async از داده ها استفاده کند.



در طراحی و توسعه ی محیط آزمایش و از زبان برنامه نویسی جاوا نسخه 21 و فریم ورک Spring Boot 3.5.4، Apache Kafka 3.9.2 و MongoDB و PostgreSQL استفاده شده است.

داده های ارسال شده در دسته های ۳۰۹۰۰۰، ۶۱۸۰۰۰ و ۹۲۷۰۰۰ تایی(به سایز ۵۰ مگابایت، ۱۰۰ مگابایت و ۱۵۰ مگابایت) به صورت جریانی و با ۴۱۶ ترکیب از ۳ پارامتر حداکثر حجم هر batch به بایت، زمان انتظار تولیدکننده به میلی‌ثانیه، نوع فشرده‌سازی پیام به کارگزار پیام ارسال می شدند و در هربار ارسال، تعداد اسناد ارسال‌شده در ثانیه از برنامه ی تولید کننده به کارگزار پیام اندازه گیری می شد. به این ترتیب هربار آزمایش منجر به تولید ۱۲۴۸ داده می شد. برای کاهش خطای اندازه گیری و سایر خطاها، این جمع آوری داده ۳ بار تکرار شد و داده ی جمع آوری شده جهت پیش پردازش آماده شد.

۳ -پیش‌پردازش داده

۳.۱ -تعریف ویژگی‌ها و متغیر هدف

پس از بارگذاری داده‌ها، ستون‌ها به دو دسته تقسیم شدند:

ویژگی‌های ورودی:(Features)

ویژگی	نوع	توضیح
batchSize	عددی	حداکثر حجم هر batch به بایت
lingerMs	عددی	زمان انتظار تولیدکننده به میلی‌ثانیه
compressionType	دسته‌ای	نوع فشرده‌سازی پیام
billSize	عددی	تعداد اسناد JSON ارسال‌شده

متغیر هدف:(Target)

متغیر	نوع	توضیح
throughput	عددی پیوسته	تعداد اسناد ارسال‌شده در ثانیه

۳.۲ -میانگین‌گیری از تکرار آزمایش‌ها

با توجه به اینکه هر آزمایش سه بار تکرار شده بود، داده‌های سه فایل JSON ادغام شده و به ازای هر ترکیب یکتا از پارامترهای تنظیماتی، میانگین مقادیر throughput محاسبه گردید. این ترکیب یکتا بر اساس چهار ستون batchSize ، lingerMs ، compressionType و billSize تعریف شد. این عملیات باعث کاهش نویز ناشی از عوامل محیطی نظیر وقفه‌هایJVM ، زمان‌بندی سیستم‌عامل و جیتر شبکه گردید.

۳.۳ -تقسیم داده به مجموعه آموزشی و آزمایشی

داده‌ها با نسبت ۸۰ به ۲۰ به مجموعه آموزشی و آزمایشی تقسیم شدند. برای اطمینان از تکرارپذیری نتایج، مقدار random_state برابر ۴۲ تنظیم گردید.

۳.٤ -کدگذاری ویژگی دسته‌ای

ویژگی compressionType که شامل چهار مقدار none، snappy، lz۴و gzip می‌باشد، با استفاده از روش One-Hot Encoding به چهار ستون باینری مجزا تبدیل گردید. ترتیب دسته‌بندی‌ها به صورت ثابت تعریف شد تا از جابجایی ستون‌ها هنگام پیش‌بینی روی داده‌های جدید جلوگیری شود.

compressionType	none	snappy	lz٤	gzip
none	۱	۰	۰	۰
snappy	۰	۱	۰	۰
lz٤	۰	۰	۱	۰
gzip	۰	۰	۰	۱

۳.۵ -نرمال‌سازی ویژگی‌های عددی

ویژگی‌های عددی batchSize، lingerMsو billSize با استفاده از StandardScaler استانداردسازی شدند. این تبدیل هر مقدار را به صورت زیر محاسبه می‌کند:

$$x' = \frac{x - \mu}{\sigma}$$

که در آن μ میانگین و σ انحراف معیار ویژگی مربوطه است. این کار از تسلط ویژگی‌هایی با دامنه عددی بزرگ‌تر مانند billSize بر ویژگی‌هایی با دامنه کوچک‌تر مانند lingerMs جلوگیری می‌کند و برای الگوریتم GPR که به مقیاس ویژگی‌ها حساس است، ضروری می‌باشد.

۳.٦ -جلوگیری از نشت داده

تمام تبدیل‌های آماری StandardScaler و OneHotEncoder تنها بر اساس داده‌های مجموعه آموزشی برازش شده و سپس بر هر دو مجموعه آموزشی و آزمایشی اعمال گردیدند. این رویکرد از نشت داده (Data Leakage) جلوگیری

می‌کند، زیرا اگر آمار مجموعه آزمایشی در محاسبه پارامترهای نرمال‌سازی استفاده می‌شد، مدل به طور غیرمستقیم اطلاعاتی از داده‌های آزمایشی در اختیار داشت.

۳.۷- نرمال‌سازی متغیر هدف برایGPR

از آنجا که الگوریتم GPR به توزیع گاوسی متغیر هدف حساس است، یک TargetScalerسفارشی پیاده‌سازی شد که مقادیر throughput را نیز استانداردسازی می‌کند. این کلاس علاوه بر تبدیل مستقیم، قابلیت معکوس‌سازی inverse transform را نیز دارد تا پیش‌بینی‌های مدل GPR از فضای نرمال‌شده به واحد واقعی تبدیل شوند.

٤-جنگل تصادفی

٤-١- مبانی نظری

جنگل تصادفی (Random Forest) یک الگوریتم یادگیری ماشین از خانواده روش‌های Bagging است که توسط Breiman در سال ۲۰۰۱ معرفی شد. ایده اصلی این الگوریتم بر اساس ترکیب تعداد زیادی درخت تصمیم مستقل بنا شده است که هر کدام روی یک زیرمجموعه تصادفی از داده‌های آموزشی آموزش می‌بینند.

در مرحله آموزش، برای هر درخت یک زیرمجموعه تصادفی از داده‌های آموزشی با جایگذاری (Bootstrap Sampling) انتخاب می‌شود. علاوه بر این، در هر گره از درخت، به جای بررسی تمام ویژگی‌ها، تنها یک زیرمجموعه تصادفی از ویژگی‌ها بررسی می‌شود. این دو منبع تصادفی بودن باعث می‌شود که درخت‌ها با یکدیگر همبستگی کمتری داشته باشند و مدل نهایی از overfitting در امان بماند.

برای یک مسئله رگرسیون، پیش‌بینی نهایی از میانگین پیش‌بینی تمام درخت‌ها به دست می‌آید:

$$\hat{y} = \frac{1}{T} \sum_{t=1}^{T} h_t(x)$$

که در آن T تعداد درخت‌ها و $h_t(x)$ پیش‌بینی درخت t ام برای ورودی x است.

معیار تقسیم هر گره در درخت‌های رگرسیون بر اساس کمینه‌سازی واریانس (Mean Squared Error) تعریف می‌شود:

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \bar{y})^2$$

که در آن y_i مقادیر واقعی و $\bar{y}$ میانگین مقادیر در آن گره است.

یکی از ویژگی‌های مهم جنگل تصادفی، قابلیت محاسبه اهمیت ویژگی‌ها (Feature Importance) است. این مقدار بر اساس میانگین کاهش ناخالصی (Mean Decrease in Impurity) در تمام درخت‌ها محاسبه می‌شود:

$$FI_j = \frac{1}{T}\sum_{t=1}^{T}\sum_{n \in N_t} p(n) \cdot \Delta I(n, j)$$

که در آن $p(n)$ نسبت نمونه‌هایی است که به گره n می‌رسند و $\Delta I(n, j)$ کاهش ناخالصی ناشی از تقسیم روی ویژگی j در گره n است.

۲-٤ -پیاده‌سازی و تنظیم پارامترها

برای پیش‌بینی throughput از کلاس MultiOutputRegressor در کتابخانه scikit-learn استفاده شد که یک مدل جنگل تصادفی مستقل برای هر متغیر هدف آموزش می‌دهد. پارامترهای مدل به شرح زیر تنظیم گردیدند:

پارامتر	مقدار	توضیح
n_estimators	۱۰۰	تعداد درخت‌های تصمیم
max_depth	None	عمق نامحدود درخت‌ها
min_samples_split	۵	حداقل نمونه لازم برای تقسیم یک گره
n_jobs	-۱	استفاده از تمام هسته‌های پردازنده
random_state	۴۲	مقدار ثابت برای تکرارپذیری نتایج

مقدار n_estimators برابر ۱۰۰ انتخاب شد که تعادل مناسبی بین دقت مدل و زمان آموزش فراهم می‌کند. مقدار min_samples_split برابر ۵ تنظیم شد تا از overfitting در برگ‌های درخت جلوگیری شود.

۳-٤ -ارزیابی مدل

برای ارزیابی عملکرد مدل از سه معیار R^2 ، MAE و RMSE استفاده شد.

ضریب تعیین R^2 میزان واریانس متغیر هدف را که توسط مدل توضیح داده می‌شود نشان می‌دهد:

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}$$

میانگین خطای مطلق (MAE) میانگین قدرمطلق تفاوت بین مقادیر واقعی و پیش‌بینی‌شده را اندازه‌گیری می‌کند:

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

ریشه میانگین مربعات خطا (RMSE) نسبت به خطاهای بزرگ حساس‌تر است:

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

نتایج ارزیابی مدل جنگل تصادفی روی مجموعه آزمایشی به شرح زیر است:

معیار	مقدار
R^2	۰.۸۶۲۳
MAE	۶۶۶۲.۶۱ docs/sec
RMSE	۱۰۴۲۰.۲۷ docs/sec

مقدار R^2 برابر ۰.۸۶۲۳ نشان می‌دهد که مدل قادر است ۸۶.۲۳ درصد از واریانس throughput را توضیح دهد که برای یک سیستم توزیع‌شده با عوامل نویز محیطی، نتیجه قابل قبولی است.

۵ –XGBoost

۵-۱- مبانی نظری

XGBoost که مخفف Extreme Gradient Boosting است، یک الگوریتم یادگیری ماشین از خانواده روش‌های Boostingاست که توسط Chen و Guestrin در سال ۲۰۱۶ معرفی شد. برخلاف جنگل تصادفی که درخت‌ها را به صورت موازی و مستقل آموزش می‌دهد، XGBoost درخت‌ها را به صورت متوالی می‌سازد و هر درخت جدید سعی می‌کند خطاهای درخت قبلی را اصلاح کند.

ایده اصلی این الگوریتم بر اساس بهینه‌سازی یک تابع هدف (Objective Function) است که از دو بخش تشکیل شده است:

$$Obj = \sum_{i=1}^{n} L(y_i, \hat{y}_i) + \sum_{t=1}^{T} \Omega(h_t)$$

که در آن بخش اول L تابع زیان (Loss Function) است که خطای پیش‌بینی را اندازه‌گیری می‌کند و بخش دوم جمله منظم‌سازی (Regularization) است که پیچیدگی مدل را کنترل می‌کند:

$$\Omega(h_t) = \gamma T + \frac{1}{2}\lambda \sum_{j=1}^{T} w_j^2$$

که در آن T تعداد برگ‌های درخت، w_j وزن هر برگ، γ جریمه تعداد برگ‌ها و λ جریمه وزن برگ‌هاست.

پیش‌بینی نهایی از جمع خروجی تمام درخت‌ها به دست می‌آید:

$$\hat{y}_i = \sum_{t=1}^{T} h_t(x_i)$$

در هر مرحله ازBoosting ، درخت جدید بر اساس گرادیان و هسیان تابع زیان نسبت به پیش‌بینی فعلی ساخته می‌شود:

$$g_i = \frac{\partial L(y_i, \hat{y}_i^{(t-1)})}{\partial \hat{y}_i^{(t-1)}}, \quad h_i = \frac{\partial^2 L(y_i, \hat{y}_i^{(t-1)})}{\partial (\hat{y}_i^{(t-1)})^2}$$

استفاده همزمان از گرادیان مرتبه اول و دوم باعث می‌شود که XGBoost نسبت به روش‌های Boosting سنتی سریع‌تر و دقیق‌تر همگرا شود.

۵.۲ پیاده‌سازی و تنظیم پارامترها

مدل XGBoost با پارامترهای زیر پیاده‌سازی شد:

پارامتر	مقدار	توضیح
n_estimators	۱۰۰	تعداد مراحلBoosting
learning_rate	۰.۱	نرخ یادگیری — سهم هر درخت در پیش‌بینی نهایی
max_depth	۶	حداکثر عمق هر درخت
subsample	۰.۸	نسبت نمونه‌های استفاده‌شده در هر مرحله
colsample_bytree	۰.۸	نسبت ویژگی‌های استفاده‌شده در هر درخت
n_jobs	−۱	استفاده از تمام هسته‌های پردازنده
random_state	۴۲	مقدار ثابت برای تکرارپذیری نتایج

مقدار learning_rate برابر ۰.۱ انتخاب شد که تعادل مناسبی بین سرعت همگرایی و دقت مدل فراهم می‌کند. مقادیر subsampleو colsample_bytree هر دو برابر ۰.۸ تنظیم شدند تا با ایجاد تصادفی بودن در فرآیند آموزش از overfittingجلوگیری شود.

۳-۵ -ارزیابی مدل

نتایج ارزیابی مدل XGBoost روی مجموعه آزمایشی به شرح زیر است:

معیار	مقدار
R^2	۰.۸۸۲۴
MAE	۶۵۸۴.۹۶ docs/sec
RMSE	۹۶۲۷.۰۰ docs/sec

مقدار R^2 برابر ۰.۸۸۲۴ نشان می‌دهد که XGBoost در مقایسه با جنگل تصادفی عملکرد بهتری داشته است. این برتری به دلیل ماهیت متوالی الگوریتم است که در آن هر درخت به طور هدفمند خطاهای درخت قبلی را کاهش می‌دهد. همچنین مقدار RMSE کمتر نسبت به جنگل تصادفی نشان‌دهنده پیش‌بینی‌های پرت کمتر در این مدل است.

٦-رگرسیون فرآیند گاوسی

۱-٦- مبانی نظری

رگرسیون فرآیند گاوسی (Gaussian Process Regression) یک روش یادگیری ماشین احتمالاتی غیرپارامتریک است که بر اساس فرآیندهای گاوسی بنا شده است. برخلاف روش‌های مبتنی بر درخت تصمیم که یک مقدار قطعی پیش‌بینی می‌کنند، GPRیک توزیع احتمال روی توابع ممکن را مدل می‌کند و علاوه بر پیش‌بینی مقدار میانگین، عدم قطعیت پیش‌بینی را نیز ارائه می‌دهد.

یک فرآیند گاوسی به صورت زیر تعریف می‌شود:

$$f(x) \sim \mathcal{GP}(m(x), k(x, x'))$$

که در آن m(x) تابع میانگین و k(x, x') تابع کوواریانس (kernel) است که شباهت بین دو نقطه x و x' را اندازه‌گیری می‌کند.

با داشتن داده‌های آموزشی X و مقادیر هدف y ، توزیع پسین (Posterior Distribution) برای نقاط آزمایشی X_* به صورت زیر محاسبه می‌شود:

$$p(f_*|X_*, X, y) = \mathcal{N}(\mu_*, \Sigma_*)$$

که در آن میانگین و واریانس پسین به صورت زیر تعریف می‌شوند:

$$\mu_* = K(X_*, X)[K(X, X) + \sigma_n^2 I]^{-1} y$$

$$\Sigma_* = K(X_*, X_*) - K(X_*, X)[K(X, X) + \sigma_n^2 I]^{-1} K(X, X_*)$$

که در آن K(X, X) ماتریس کوواریانس داده‌های آموزشی، $K(X_*, X)$ماتریس کوواریانس بین داده‌های آزمایشی و آموزشی و σ_n^2 واریانس نویز اندازه‌گیری است.

در این پژوهش از kernel ترکیبی زیر استفاده شد:

$$k(x, x') = C \cdot k_{Matern}(x, x') + k_{White}(x, x')$$

که در آن C یک ثابت مقیاس(ConstantKernel) ، k_{Matern}kernel ماترن با پارامتر nu = ۱.۵ و k_{White} kernel سفید برای مدل‌سازی نویز اندازه‌گیری است.

kernel ماترن با nu = ۱.۵ به صورت زیر تعریف می‌شود:

$$k_{Matern}(r) = \left(1 + \frac{\sqrt{3}r}{l}\right) \exp\left(-\frac{\sqrt{3}r}{l}\right)$$

که در آن $r = ||x - x'||$ فاصله اقلیدسی بین دو نقطه و l پارامتر طول مقیاس (length scale) است. انتخاب این kernel نسبت به kernel گاوسی (RBF) مناسب‌تر است زیرا فرض صافی کمتری در مورد رابطه بین پارامترهای Kafka و throughput دارد.

۶-۲- پیاده‌سازی و تنظیم پارامترها

مدل GPR با پارامترهای زیر پیاده‌سازی شد:

پارامتر	مقدار	توضیح
kernel	ConstantKernel × Matern(ν=۱.۵) + WhiteKernel	ترکیب kernel برای مدل‌سازی رفتار غیرخطی و نویز
n_restarts_optimizer	۱۰	تعداد دفعات بهینه‌سازی پارامترهای kernelاز نقاط شروع مختلف
normalize_y	True	نرمال‌سازی داخلی متغیر هدف
random_state	۴۲	مقدار ثابت برای تکرارپذیری نتایج

مقدار n_restarts_optimizer برابر ۱۰ انتخاب شد تا از گیر افتادن بهینه‌ساز در مینیمم‌های محلی جلوگیری شود. همچنین از آنجا که GPR به مقیاس متغیر هدف حساس است، علاوه بر normalize_y داخلی، یک TargetScaler سفارشی نیز برای استانداردسازی مقادیر throughput پیش از ورود به مدل به کار گرفته شد.

۶.۳ ارزیابی مدل

نتایج ارزیابی مدل GPR روی مجموعه آزمایشی به شرح زیر است:

معیار	مقدار
R^2	۰.۸۲۱۱
MAE	۷۵۶۵.۱۳ docs/sec
RMSE	۱۱۸۷۶.۰۴ docs/sec

هرچند GPR در مقایسه با سایر مدل‌های منفرد، R^2 کمتری به دست آورد، مزیت آن امکان برآورد عدم‌قطعیت پیش‌بینی است. میانگین عدم‌قطعیت گزارش‌شده برابر ۰.۳۴۸۲ در فضای مقیاس‌بندی‌شده هدف است و نباید به عنوان خطا با واحد سند بر ثانیه تفسیر شود. این قابلیت مربوط به خود مدل GPR است و در خروجی فعلی تابع پیشنهاددهنده Stacking نمایش داده نمی‌شود.

۷- LightGBM

۷-۱ -مبانی نظری

LightGBM که مخفف Light Gradient Boosting Machine است، یک الگوریتم یادگیری ماشین از خانواده روش‌های Boosting است که توسط Microsoft در سال ۲۰۱۷ معرفی شد. این الگوریتم بر پایه همان اصول XGBoost بنا شده است، اما با دو نوآوری اساسی سرعت آموزش و کارایی حافظه را به طور قابل توجهی بهبود می‌بخشد.

نوآوری اول، روش رشد درخت Leaf-wise است. برخلاف XGBoost که درخت را سطح به سطح (Level-wise) می‌سازد، LightGBM در هر مرحله برگی را انتخاب می‌کند که بیشترین کاهش در تابع زیان را ایجاد می‌کند:

$$leaf^* = \arg\max_{leaf} \Delta L(leaf)$$

که در آن $\Delta L(leaf)$ کاهش تابع زیان ناشی از تقسیم برگ مورد نظر است. این رویکرد باعث می‌شود که با همان تعداد برگ، درخت‌های عمیق‌تر و دقیق‌تری نسبت به روش Level-wise ساخته شود.

نوآوری دوم، الگوریتم GOSS یا Gradient-based One-Side Sampling است. در این روش به جای استفاده از تمام نمونه‌های آموزشی در هر مرحله، نمونه‌هایی با گرادیان بزرگ (نمونه‌هایی که مدل روی آن‌ها خطای بیشتری دارد) با وزن بیشتر انتخاب می‌شوند:

$$\tilde{g}_i = \begin{cases} g_i & \text{if } |g_i| \geq \text{threshold} \\ \frac{1-a}{b} g_i & \text{otherwise} \end{cases}$$

که در آن a نسبت نمونه‌های با گرادیان بزرگ و b نسبت نمونه‌های با گرادیان کوچک است که به صورت تصادفی انتخاب می‌شوند.

تابع هدف LightGBM مشابه XGBoost است و از گرادیان مرتبه اول و دوم تابع زیان استفاده می‌کند:

$$Obj^{(t)} = \sum_{i=1}^{n} \left[g_i h_t(x_i) + \frac{1}{2} h_i h_t^2(x_i) \right] + \Omega(h_t)$$

که در آن g_i و h_i به ترتیب گرادیان و هسیان تابع زیان در مرحله t ام هستند.

۲-۷- پیاده‌سازی و تنظیم پارامترها

مدل LightGBM با پارامترهای زیر پیاده‌سازی شد:

پارامتر	مقدار	توضیح
n_estimators	۱۰۰	تعداد مراحل Boosting
learning_rate	۰.۱	نرخ یادگیری — سهم هر درخت در پیش‌بینی نهایی
max_depth	۶	حداکثر عمق هر درخت
subsample	۰.۸	نسبت نمونه‌های استفاده‌شده در هر مرحله
colsample_bytree	۰.۸	نسبت ویژگی‌های استفاده‌شده در هر درخت
n_jobs	−۱	استفاده از تمام هسته‌های پردازنده
random_state	۴۲	مقدار ثابت برای تکرارپذیری نتایج

۳-۷- ارزیابی مدل

نتایج ارزیابی مدل LightGBM روی مجموعه آزمایشی به شرح زیر است:

معیار	مقدار
R^2	۰.۸۵۵۶
MAE	۷۳۱۴.۱۷ docs/sec
RMSE	۱۰۶۶۷.۸۷ docs/sec

مقدار R^2 برابر ۰.۸۵۵۶ نشان می‌دهد که LightGBM عملکردی نزدیک به جنگل تصادفی داشته است. در بررسی همبستگی بین پیش‌بینی‌های XGBoost وLightGBM ، مقدار ضریب همبستگی برابر ۰.۹۸۶۰ به دست آمد که نشان‌دهنده شباهت بسیار زیاد خروجی این دو مدل است. این موضوع به دلیل ماهیت مشابه هر دو الگوریتم Gradient و Boosting استفاده از پارامترهای یکسان است و در مرحله Stacking اثرات مستقیمی داشت که در بخش بعد توضیح داده می‌شود.

Stacking-۸

۸-۱ -مبانی نظری

Stacking یا Stacked Generalization یک روش یادگیری ترکیبی (Ensemble Learning) است که توسط Wolpertدر سال ۱۹۹۲ معرفی شد. برخلاف Bagging و Boosting که از مدل‌های همنوع استفاده می‌کنند، Stacking چندین مدل ناهمگون را با یکدیگر ترکیب می‌کند و یک مدل سطح بالاتر (Meta-Model) را آموزش می‌دهد تا بهترین ترکیب از پیش‌بینی‌های مدل‌های پایه را بیاموزد.

معماری کلی Stacking از دو لایه تشکیل شده است. در لایه اول، مدل‌های پایه (Base Models) مستقیماً روی داده‌های آموزشی آموزش می‌بینند و پیش‌بینی‌هایشان به عنوان ویژگی‌های جدید (Meta-Features) استفاده می‌شود. در لایه دوم، Meta-Modelروی این Meta-Features آموزش می‌بیند تا پیش‌بینی نهایی را تولید کند:

$$\hat{y}_{stack} = f_{meta}(\hat{y}_{RF}, \hat{y}_{XGB}, \hat{y}_{GPR})$$

که در آن Meta-Model f_{meta} و $\hat{y}RF$ ، $\hat{y}XGB$ و $\hat{y}GPR$ پیش‌بینی‌های مدل‌های پایه هستند.

برای جلوگیری از نشت داده در ساخت Meta-Features ، از روش K-Fold Cross Validation استفاده شد. در این روش برای هرfold ، مدل‌های پایه روی داده‌های خارج از آن fold آموزش دیده و پیش‌بینی را روی داده‌های داخل آن fold انجام می‌دهند:

$$\hat{y}_i^{(k)} = h_k(x_i), \quad x_i \in Fold_k, \quad h_k \text{ trained on } X \setminus Fold_k$$

این رویکرد تضمین می‌کند که Meta-Model روی پیش‌بینی‌هایی آموزش می‌بیند که مدل‌های پایه در زمان تولید آن‌ها، آن نمونه‌ها را ندیده‌اند.

Meta-Model مورد استفاده در این پژوهش رگرسیون Ridge است که تابع هدف آن به صورت زیر تعریف می‌شود:

$$\min_w \sum_{i=1}^{n} (y_i - w^T \hat{y}_i)^2 + \alpha ||w||^2$$

که در آن w وزن هر مدل پایه، $\hat{y}_i$ بردار Meta-Features و α پارامتر منظم‌سازی است. انتخاب Ridge به عنوان Meta-Model به این دلیل است که با تنها سه ورودی (پیش‌بینی سه مدل پایه)، مدل ساده‌ای لازم است که از overfitting جلوگیری کند.

۲-۸- انتخاب مدل‌های پایه

ابتدا چهار مدل جنگل تصادفی، GPR ،XGBoost و LightGBM برای Stacking بررسی شدند. ضریب همبستگی ۰.۹۸۶۰ میان پیش‌بینی‌های XGBoost و LightGBM نشان می‌دهد که خروجی‌های آن‌ها بسیار مشابه است. در آزمایش اولیه چهارمدلی، ضریب LightGBM در فرامدل Ridge منفی بود؛ بااین‌حال، همبستگی بالا یا ضریب منفی به‌تنهایی اثبات‌کننده بهبود عملکرد پس از حذف یک مدل نیست. با هدف کاهش پیچیدگی و ارزیابی تجربی یک ترکیب متنوع‌تر، LightGBM از مدل‌های پایه نسخه منتخب حذف شد و سه مدل زیر باقی ماندند:

مدل	نوع	دلیل انتخاب

Random Forest	Bagging	تنوع ساختاری، مقاوم در برابر نویز
XGBoost	Boosting	بهترین عملکرد در بین مدل‌های تکی
GPR	Probabilistic	ماهیت متفاوت از دو مدل دیگر، ارائه عدم قطعیت

۳-۸ -پیاده‌سازی و تنظیم پارامترها

پیاده‌سازی Stacking در چهار مرحله انجام شد:

مرحله اول، ایجاد سه ستون Meta-Features از پیش‌بینی‌های خارج از Fold مدل‌های Random Forest، XGBoost و GPR با روش اعتبارسنجی متقاطع پنج‌بخشی (۵-Fold Cross-Validation) بر داده‌های آموزشی بود. در هر مرحله، مدل‌ها بر چهار بخش آموزش دیده و برای بخش پنجم پیش‌بینی تولید کردند.

مرحله دوم، آموزش Meta-Model بود Ridge Regression .با پارامتر منظم‌سازی $\alpha = ۱.۰$ روی Meta-Features ساخته‌شده آموزش دید.

مرحله سوم، تولید پیش‌بینی برای مجموعه آزمایشی با سه مدل پایه آموزش‌دیده روی کل مجموعه آموزش بود. پیش‌بینی GPR پس از تبدیل معکوس مقیاس هدف، به واحد واقعی throughput بازگردانده شد تا تمام فراویژگی‌ها واحد یکسانی داشته باشند.

مرحله چهارم، پیش‌بینی نهایی بود Meta-Model .پیش‌بینی‌های سه مدل پایه را ترکیب کرده و مقدار نهایی throughput را تولید کرد.

در نسخه اصلاح‌شده، فرامدل Ridge فقط سه فراویژگی به ترتیب پیش‌بینی‌های XGBoost، Random Forest و GPR را دریافت می‌کند. به‌همین دلیل، ضرایب Ridge چهارمدلی پیشین برای این مدل سه‌مدلی قابل استفاده نیستند.

ضرایب سه‌مدلی و عرض از مبدأ Ridge در خروجی معیارهای ارزیابی ارسالی ثبت نشده‌اند؛ ازاین‌رو، برای پرهیز از انتساب نادرست ضرایب نسخه چهارمدلی به نسخه نهایی، جدول وزن‌های قبلی از این گزارش حذف شده است. تحلیل عملکرد بر نتایج مجموعه آزمونِ نسخه سه‌مدلی استوار است.

٤-٨- ارزیابی مدل

نتایج ارزیابی مدل Stacking روی مجموعه آزمایشی به شرح زیر است:

معیار	مقدار
R^2	۰.۸۸۵۱
MAE	۶۳۸۶.۲۸ docs/sec
RMSE	۹۵۱۸.۶۳ docs/sec

مدل Stacking سه‌مدلی با R^2 برابر ۰.۸۸۵۱ بالاترین مقدار این معیار را در میان پنج مدل ارزیابی‌شده به دست آورد. در مقایسه با بهترین مدل منفرد، یعنی XGBoost با R^2 برابر ۰.۸۸۲۴، مقدار MAE از ۶۵۸۴.۹۶ به ۶۳۸۶.۲۸ و RMSE از ۹۶۲۷.۰۰ به ۹۵۱۸.۶۳ سند بر ثانیه کاهش یافت. این بهبود در مجموعه آزمون محدود است و بدون تکرار آزمایش‌ها یا آزمون آماری نباید «معنادار» قلمداد شود.

۹- نتایج

۹-۱ -مقایسه عملکرد مدل‌ها

در این پژوهش پنج مدل یادگیری ماشین برای پیش‌بینی توان عملیاتی (throughput) سیستم Kafka مورد ارزیابی قرار گرفتند. جدول زیر نتایج ارزیابی تمام مدل‌ها را روی مجموعه آزمایشی نشان می‌دهد:

مدل	R^2	MAE (docs/sec)	RMSE (docs/sec)
Random Forest	۰.۸۶۲۳	۶۶۶۲.۶۱	۱۰۴۲۰.۲۷
XGBoost	۰.۸۸۲۴	۶۵۸۴.۹۶	۹۶۲۷.۰۰
GPR	۰.۸۲۱۱	۷۵۶۵.۱۳	۱۱۸۷۶.۰۴
LightGBM	۰.۸۵۵۶	۷۳۱۴.۱۷	۱۰۶۶۷.۸۷
Stacking (سه مدل پایه)	۰.۸۸۵۱	۶۳۸۶.۲۸	۹۵۱۸.۶۳

۹-۲ -تحلیل نتایج

با بررسی نتایج جدول بالا، چند نکته قابل توجه است.

Stacking با سه مدل پایه، R^2 برابر ۰.۸۸۵۱ و کمترین مقدار MAE و RMSE را در میان پنج مدل این ارزیابی دارد. بهبود آن نسبت به XGBoost اندک است؛ بنابراین، نتایج از برتری محدود در همین مجموعه آزمون پشتیبانی می‌کنند و به‌تنهایی اثبات‌کننده افزایش توان عملیاتی واقعی پس از استقرار نیستند.

برای بررسی اثر حذف LightGBM، خروجی ثبت‌شده نسخه آزمایشی چهارمدلی نیز به‌عنوان مرجع تاریخی مقایسه شد: R^2 برابر ۰.۸۸۴۹، MAE برابر ۶۳۵۰.۲۲ و RMSE برابر ۹۵۲۷.۰۹ سند بر ثانیه. نسخه سه‌مدلی به‌ترتیب مقادیر ۰.۸۸۵۱، ۶۳۸۶.۲۸ و ۹۵۱۸.۶۳ را به دست آورد؛ بنابراین R^2 و RMSE اندکی بهتر شدند، درحالی‌که MAE کمی افزایش یافت. کاهش پیچیدگی لزوماً بهبود همه معیارها را به همراه نداشته است.

XGBoost با R^2 برابر ۰.۸۸۲۴ در رتبه دوم قرار گرفت و از نظر عملکرد بسیار نزدیک به Stacking است. برتری این مدل نسبت به جنگل تصادفی و LightGBM به دلیل ماهیت متوالی الگوریتم Boosting است که در آن هر درخت به طور هدفمند خطاهای قبلی را اصلاح می‌کند.

جنگل تصادفی با R^2 برابر ۰.۸۶۲۳ در رتبه سوم قرار گرفت و به عنوان یک مدل پایه قوی و پایدار عمل کرد. این مدل علی‌رغم سادگی نسبی، عملکردی نزدیک به XGBoost و Stacking داشت.

LightGBMبا R^2 برابر ۰.۸۵۵۶ عملکردی مشابه جنگل تصادفی داشت. همبستگی بالای ۰.۹۸۶۰ بین پیش‌بینی‌های LightGBMو XGBoost نشان داد که این دو مدل اطلاعات مشابهی از داده استخراج می‌کنند و به همین دلیل LightGBMاز مجموعه مدل‌های پایه Stacking حذف گردید.

برابر ۰.۸۲۱۱ کمترین عملکرد پیش‌بینی را در میان چهار مدل مستقل داشت؛ درعین‌حال، به‌دلیل ماهیت R^2 با GPR احتمالاتی، قابلیت محاسبه عدم‌قطعیت پیش‌بینی را فراهم می‌کند. این قابلیت می‌تواند در توسعه‌های بعدی پیشنهاددهنده .مقدار عدم‌قطعیت به کاربر نمایش داده نمی‌شود recommend_stacking مفید باشد، اما در خروجی فعلی تابع

۹-۳ -سیستم پیشنهاددهنده

بر اساس مدل Stacking به عنوان بهترین مدل، یک سیستم پیشنهاددهنده پیاده‌سازی شد. این سیستم با دریافت تعداد اسناد JSON مورد نظر کاربر(billSize) ، تمام ۴۱۶ ترکیب ممکن از پارامترهای تنظیماتی Kafka را ارزیابی کرده و بهترین

تنظیمات را بر اساس حداکثر throughput پیش‌بینی‌شده پیشنهاد می‌دهد. این ۴۱۶ ترکیب از حاصل‌ضرب مقادیر زیر به دست می‌آید:

پارامتر	تعداد مقادیر
batchSize	۴
lingerMs	۲۶
compressionType	۴
جمع	۴ × ۲۶ × ۴ = ۴۱۶

۱۰- نتیجه‌گیری

در این پژوهش یک سیستم پیشنهاددهنده مبتنی بر یادگیری ماشین برای بهینه‌سازی پارامترهای تنظیماتی Apache Kafka طراحی و پیاده‌سازی شد. هدف اصلی این سیستم، پیش‌بینی توان عملیاتی (throughput) و پیشنهاد بهترین ترکیب از پارامترهای batchSize، lingerMsو compressionType برای یک حجم مشخص از داده‌های ورودی بود.

در مرحله جمع‌آوری داده، آزمایش‌ها روی یک محیط واقعی متشکل از سرور Apache Kafka و یک برنامه تولیدکننده پیام پیاده‌سازی‌شده با Java ۲۱ و Spring Boot ۳.۵.۴ انجام شد. در مجموع ۴۱۶ ترکیب تنظیماتی با سه حجم مختلف از داده آزمایش شدند و هر آزمایش سه بار تکرار گردید تا اثرات نویز محیطی کاهش یابد.

در مرحله پیش‌پردازش، داده‌های سه اجرا برای هر ترکیب تنظیماتی میانگین‌گیری شدند. ویژگی‌های عددی با استفاده از StandardScaler نرمال‌سازی شده و ویژگی دسته‌ای compressionType با روش One-Hot Encoding به چهار ستون باینری تبدیل گردید. تمام تبدیل‌ها تنها بر اساس داده‌های آموزشی برازش شدند تا از نشت داده جلوگیری شود.

چهار الگوریتم یادگیری ماشین شامل جنگل تصادفی، XGBoost، GPR و LightGBM آموزش دیده و ارزیابی شدند. نتایج نشان داد که XGBoost با R^2 برابر ۰.۸۸۲۴ بهترین عملکرد را در میان مدل‌های تکی داشت. در مرحله بعد، روش Stacking با استفاده از جنگل تصادفی، XGBoost و GPR به عنوان مدل‌های پایه و Ridge Regression به عنوان

Meta-Model پیاده‌سازی شد. LightGBM به دلیل همبستگی بسیار بالا (۰.۹۸۶۰) با XGBoost از مجموعه مدل‌های پایه Stacking حذف گردید. مدل Stacking با R^2 برابر ۰.۸۸۵۱، MAE برابر ۶۳۸۶.۲۸ داده بر ثانیه و RMSE برابر ۹۵۱۸.۶۳ داده بر ثانیه بهترین عملکرد کلی را به دست آورد.

در نهایت سیستم پیشنهاددهنده بر اساس مدل Stacking پیاده‌سازی شد که با دریافت تعداد اسناد مورد نظر کاربر، تمام ۴۱۶ ترکیب ممکن از پارامترهای تنظیماتی را ارزیابی کرده و بهترین تنظیمات را برای حداکثر کردن throughput پیشنهاد می‌دهد.

همچنین اضافه کردن پارامترهای تنظیماتی بیشتر Kafka نظیر acks، buffer.memory و max.in.flight.requests.per.connection می‌تواند دقت سیستم پیشنهاددهنده را بهبود بخشد.

منابع و مآخذ

[۱] M. A. R. A. „ S. M. T. a. T. C. Md Showkat Hossain Chy, "Comparative Evaluation of Java Virtual Machine-Based Message Queue Services: A Study on Kafka, Artemis, Pulsar and RocketMQ".

[۲] S. K. ·. Y. B. ·. R. C. ·. D. E. Houssaini, "Design and implementation of a cloud-based event-driven architecture for real-time data processing in wireless sensor networks".

[۳] A. S. Kushwaha, "Stream processing with Apache Kafka pipelines".

[۴] H. M. Mohammad Amin Mohammadi Tabar, "Analysis And Evaluation of Microservices Architecture Performance in Inter-service Communication for Intelligent and Real-time Systems".